

# Additional format specifiers for `time_point`

Document #: P2945R1 [\[Latest\]](#) [\[Status\]](#)  
Date: 2024-09-09  
Project: Programming Language C++  
Audience: LEWG  
Reply-to: Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>

## Contents

---

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Why More Specifiers](#)
  - [3.1 The Capture Problem](#)
  - [3.2 Why not use precision?](#)
  - [3.3 Existing Practice in Other Languages](#)
    - [3.3.1 UNIX](#)
    - [3.3.2 C](#)
    - [3.3.3 Python](#)
    - [3.3.4 Rust](#)
    - [3.3.5 Ruby](#)
    - [3.3.6 C++ Logging Libraries](#)
    - [3.3.7 Standard C++, `<chrono>`, and `<format>`](#)
- [4 Proposal](#)
  - [4.1 `%s`, `%S`, and `%T`](#)
  - [4.2 `%f`](#)
  - [4.3 Exotic durations](#)
  - [4.4 `%Q` and `%q`](#)
  - [4.5 Implementation Experience](#)
- [5 Wording](#)
  - [5.1 The `%f` specifier](#)
  - [5.2 The `%Q` and `%q` specifiers](#)
  - [5.3 The `%S`, `%s`, and `%T` specifiers](#)
  - [5.4 Feature-test Macro](#)
- [6 References](#)

## § 1 Revision History

---

From [\[P2945R0\]](#) to R1: Removing any option to change the meaning of existing code. The current proposal is a pure extension.

## § 2 Introduction

---

`std::chrono::time_point` has a lot of format specifiers. You can peruse the table [here](#). This makes it very convenient for the user to format their `time_point` however they want: the date in either the correct (`2023-07-08`) or incorrect

(07/08/2023) numeric formats, spelling out the month (July or Jul), presenting the time in 24-hour notation or using AM or PM, etc. This is all very useful.

But there's a few format specifier that I believe are missing that I would like to propose:

| desired output      | proposed                                      | current workaround                                                                                                            |
|---------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| 1688830834295314673 | <code>std::format("{:%s}", tp)</code>         | <code>std::format("{:%Q}", tp.time_since_epoch())</code><br>or <code>std::format("{%}", tp.time_since_epoch().count())</code> |
| 15:40:34            | <code>std::format("{:%H:%M:%.0S}", tp)</code> | <code>std::format("{:%H:%M:%S}", std::chrono::time_point_cast&lt;std::chrono::seconds&gt;(tp))</code>                         |
| 15:40:34.295        | <code>std::format("{:%H:%M:%.3S}", tp)</code> | <code>std::format("{:%H:%M:%S}", std::chrono::time_point_cast&lt;std::chrono::milliseconds&gt;(tp))</code>                    |
| 15:40:34.295314     | <code>std::format("{:%H:%M:%.6S}", tp)</code> | <code>std::format("{:%H:%M:%S}", std::chrono::time_point_cast&lt;std::chrono::microseconds&gt;(tp))</code>                    |

In addition to proposing `%.nS` (for seconds with  $n$  decimal digits), this paper also proposes `%.nT` to mean `%H:%M:%.nS`.

## § 3 Why More Specifiers

First, it is simply much more convenient for the user to write something like `%.0T` if what they want is `15:40:34` then it is for them to write that rather verbose cast expression to convert their `time_point` into seconds duration.

Second, `%.0T` ensures that they don't actually have to care about the underlying duration of their `time_point`, this will consistently produce the same output regardless of whether it's `time_point<system_clock, seconds>` or `time_point<system_clock, milliseconds>` or `time_point<system_clock, nanoseconds>`.

Third, specifiers can nest in a way that those workarounds don't. For example, it is straightforward to implement formatting for `Optional<T>` such that it supports all of `T`'s format specifiers, so that `Optional<int>(42)` can format using `{:#x}` as `Some(0x2a)`. If `time_point` has the necessary specifiers then an `Optional<time_point>` can be formatted as desired simply using `time_point`'s specifiers. Otherwise, the workaround requires calling `Optional::transform`. The same argument can be made for ranges: use the underlying specifier, or have to resort to using `std::views::transform`. This certainly becomes inconvenient for the user pretty rapidly, but it also brings up a question of safety - something I'm going to call the capture problem [1](#).

Fourth, the user may only even have access to provide the specifier and may not even have the ability to touch the `time_point` *at all*. This is the case in some logging libraries (like `spdlog`) where the user-facing API can provide a specifier for how the message is formatted, but never even sees the `time_point` object.

### § 3.1 The Capture Problem

Consider:

```
void print_names(std::vector<std::string> const& names) {  
    std::println("{} ", names);  
}
```

Here, the formatting is all done immediately. `std::println` doesn't need to (and doesn't) copy names and there are no lifetime issues here at all. Now consider a slight variation:

```
void log_names(std::vector<std::string> const& names) {
    LOG("{} ", names);
}
```

Here, LOG is a stand-in for your favorite logging framework. Some of these do foreground logging (and so wouldn't need to copy names), some of these do background logging (and so would have to, and do, copy names), some of these do it conditionally. Either way, the above still works, because the logger simply does the right thing with the object.

Let's say we don't want to log the names, but rather want to log some... transformed version thereof:

```
void log_transformed_names(std::vector<std::string> const& names) {
    LOG("{} ", names | std::views::transform(f));
}
```

One example of this being wanting to provide a custom delimiter, which currently doesn't exist as a specifier:

```
void log_transformed_names(std::vector<std::string> const& names) {
    LOG("{} ", fmt::join(" and ", names));
}
```

Now we have a problem. If LOG is doing background logging (whether always or conditionally) and it copies the result of `fmt::join` and that's... just a view. We're not copying the underlying names and now we have a potential problem with lifetimes. This could now dangle.

Or it could be fine, if we're doing foreground logging! So we have this problem where if we're doing foreground logging, we definitely want to just log the view without any additional work. Whereas if we're doing background logging, we probably want to eagerly construct a vector out of it (or eagerly do the formatting for this argument) to avoid any lifetime issues.

We have no way of "just doing the right thing" here. We have no way using the view if that's good enough or collecting the elements otherwise, nor do we have a way of signaling when using the view might dangle.

Note that this is not specific to ranges and views at all, I'm just using it as a simple example.

Allowing more common logic into the specifiers simply avoids this problem. We don't have to figure out how and when to wrap arguments, since the logic entirely lives in the specifier. Here, I'm not trying to solve the general capture problem, I'm only trying to help a little bit more in the context of `time_points`.

## § 3.2 Why not use precision?

Rather than having a prefixed specifier to do millisecond precision, like `%3S` or `%.3S`, could we use the already-existing *precision* specifier and write something like `{:.3%H:%M:%S}`? We could, but I think this would be a poor choice.

First, this is what the standard has to say about *precision* for a *chrono-format-spec* (in 29.12 [\[time.format\]/1](#)):

- <sup>1</sup> [...] Giving a *precision* specification in the *chrono-format-spec* is valid only for types that are specializations of `std::chrono::duration` for which the nested *typedef-name* rep denotes a floating-point type. For all other types, an exception of type `format_error` is thrown if the *chrono-format-spec* contains a *precision* specification.

That's it. So, first, *precision* is only meaningful for floating-point types, which makes it useless here because even `double` doesn't have enough precision for nanoseconds, so `system_clock::rep` is practically speaking going to be an integral type (even though all we specify about it is that it's signed). Second, we don't... actually say what *precision* does anywhere.

But even assuming that it does what we might think of as “the obvious thing”, consider the difference between the two choices of specifier:

| using <i>precision</i>    | modifying %S              |
|---------------------------|---------------------------|
| "{: .3%Y-%m-%d %H:%M:%S}" | "{: %Y-%m-%d %H:%M:%.3S}" |

The version on the left just has this weirdly dangling `.3`, that only applies to the much later `%S`. That’s not really how any of the other specifiers behave and is needlessly harder to understand. It would also meant that you can provide a *precision* without providing any specifier that makes use of it, which is just a pointless thing to allow.

## § 3.3 Existing Practice in Other Languages

### § 3.3.1 UNIX

In the UNIX `date` program, `%S` is always an integer number of seconds and `%s` is seconds since epoch. If you want decimal digits, you can add a width to `%N` (which defaults to 9):

| specifier            | example                          |
|----------------------|----------------------------------|
| <code>+%T</code>     | <code>09:40:34</code>            |
| <code>+%T.%N</code>  | <code>09:40:34.295314673</code>  |
| <code>+%T.%3N</code> | <code>09:40:34.295</code>        |
| <code>+%S</code>     | <code>1688830834</code>          |
| <code>+%S%N</code>   | <code>1688830834295314673</code> |

### § 3.3.2 C

`tm` has no subsecond field, but in `strptime`, `%S` is an integral number of seconds and `%s` is seconds since epoch.

### § 3.3.3 Python

In `datetime.strptime`, `%S` is always an integer number of seconds and `%s` is seconds since epoch. `%f` exists to print microseconds, as in `%S.%f` or `%s%f`, but there is no way to get any other precision.

### § 3.3.4 Rust

In Rust in the [chrono crate](#), `%S` is always an integer number of seconds, `%s` is seconds since epoch. The following specifiers exist to get subsecond precision:

| specifier         | example                 |
|-------------------|-------------------------|
| <code>%f</code>   | <code>026490000</code>  |
| <code>%.f</code>  | <code>.026940</code>    |
| <code>%.3f</code> | <code>.026</code>       |
| <code>%.6f</code> | <code>.026490</code>    |
| <code>%.9f</code> | <code>.026490000</code> |
| <code>%3f</code>  | <code>026</code>        |
| <code>%6f</code>  | <code>026490</code>     |
| <code>%9f</code>  | <code>026490000</code>  |

### § 3.3.5 Ruby

In [Ruby](#), `%S` is always an integer number of seconds, `%s` is seconds since epoch, and the following specifiers can give you subsecond precision:

| specifier         | example                  |
|-------------------|--------------------------|
| <code>%L</code>   | 323                      |
| <code>%N</code>   | 323091400                |
| <code>%3N</code>  | 323                      |
| <code>%6N</code>  | 323091                   |
| <code>%9N</code>  | 323091400                |
| <code>%24N</code> | 323091400000000000000000 |

I feel like yoctoseconds is probably not a unit people are going to use very often, but there it is.

### § 3.3.6 C++ Logging Libraries

Despite broadly using `{fmt}`, [spdlog](#) actually uses its own approach for `time_point` formatting specifically to allow full control over the timestamp. As I mentioned [earlier](#), `spdlog` never exposes the `time_point` — only the ability to format it with a custom specifier. While it also uses `%S` as an integer number of seconds, it instead uses `%E` as seconds since epoch. For the fractional part, it provides distinct specifiers for milliseconds, microseconds, and nanoseconds:

| specifier       | example   |
|-----------------|-----------|
| <code>%e</code> | 678       |
| <code>%f</code> | 056789    |
| <code>%F</code> | 256789123 |

Similarly, [Quill](#), uses `strftime` as its model and thus has `%S` as two-digit integral seconds and `%s` as seconds since epoch, and additionally provides `%Qms`, `%Qus`, and `%Qns` for the fractional parts.

### § 3.3.7 Standard C++, `<chrono>`, and `<format>`

As you can see above, there’s a pretty impressive consensus on what a few specifiers mean. To everybody:

- `%S` is a specifier that gives you a two-digit, integer number of seconds from 00 to 59, and
- `%s` is a specifier that gives you the integer number of seconds since epoch (everyone except `spdlog`, which uses `%E` for this).

To everyone, that is, except C++20’s approach to chrono formatting. Why is that? Our wording comes from [\[P1361R2\]](#), which itself comes from [\[P0355R7\]](#). That paper, since R0, has always defined `%S` in the same way. The specific words changed over time, but even [\[P0355R0\]](#) states:

- If `%S` or `%T` appears in the format string and the argument `tp` has precision finer than seconds, then seconds are formatted as a decimal floating point number with a fixed format and a precision matching that of the precision of `tp`. The character for the decimal point is localized according to the `locale`.

P0355 describes itself as proposing “`strftime`-like formatting” but offers no explanation that I can find for why it differs from `strftime` in this case [2](#). Nor can I find any evidence that this difference was noted in either LEWG or LWG any of the times this paper was discussed.

I consider this an unfortunate design choice, but it’s the one we have.

## § 4 Proposal

This paper proposes that we:

- Add several new specifiers (`%s`, `%f`, `%Q`, and `%q`)
- Add modifiers to some existing ones (`%S` and `%T`)

I'll go through these in turn.

### § 4.1 `%s`, `%S`, and `%T`

I propose that we:

- Add `%s` to be the number of ticks since epoch in the `time_point`'s units.
- Allow both `%S` and `%s` to be prefixed with a precision to indicate how many subsecond digits to include. For formatting milliseconds, this would look like `%.3S` for the former and `%.3s` for the latter.
- Extend `%T` in the same way that we extend `%S`, so that `%.9T` means `%H:%M:%.9S`.

Proposed examples (which assume that `system_clock::time_point` has nanosecond resolution):

| desired output      | proposed                                      |
|---------------------|-----------------------------------------------|
| 1688830834295314673 | <code>std::format("{:%s}", tp)</code>         |
| 1688830834          | <code>std::format("{:%.0s}", tp)</code>       |
| 1688830834.295      | <code>std::format("{:%.3s}", tp)</code>       |
| 15:40:34            | <code>std::format("{:%H:%M:%.0S}", tp)</code> |
| 15:40:34.295        | <code>std::format("{:%H:%M:%.3S}", tp)</code> |
| 15:40:34.295314     | <code>std::format("{:%H:%M:%.6S}", tp)</code> |

### § 4.2 `%f`

`%f` is a new specifier that gives you the fractional seconds, with or without the decimal point, with customizable precision. The meaning for `%f`, specifically, is as follows:

|                   | precision           |                     |                         |
|-------------------|---------------------|---------------------|-------------------------|
| specifier         | seconds(1)          | milliseconds(1234)  | nanoseconds(1234567)    |
| <code>%.f</code>  | empty string        | <code>.234</code>   | <code>.001234567</code> |
| <code>%.0f</code> | empty string        | empty string        | empty string            |
| <code>%.3f</code> | <code>.000</code>   | <code>.234</code>   | <code>.001</code>       |
| <code>%.5f</code> | <code>.00000</code> | <code>.23400</code> | <code>.00123</code>     |
| <code>%f</code>   | empty string        | <code>234</code>    | <code>001234567</code>  |
| <code>%0f</code>  | empty string        | empty string        | empty string            |
| <code>%3f</code>  | <code>000</code>    | <code>234</code>    | <code>001</code>        |
| <code>%5f</code>  | <code>00000</code>  | <code>23400</code>  | <code>00123</code>      |

### § 4.3 Exotic durations

There's one more thing that needs to be touched on here. For `sys_time<nanoseconds>`, it's pretty clear what `%s` should mean (nanoseconds since epoch) and what `%.0s` would mean (seconds since epoch). But nanoseconds (and similar

units like microseconds or seconds) aren't the only durations. We also have to consider other ones.

The next most obvious one to consider is... everyone say it with me now... [microfortnights](#).

A microfortnight is, as the name suggests, one millionth of a fortnight - which is 14 days. In more familiar units, a microfortnight is equal to 1.2096 seconds.

Following the rules that we have today, formatting 1 microfortnight using %S would yield 01.2096. Or, to pick a more interesting time point that is more than a minute since epoch, formatting 123 microfortnights (148.7808 seconds) using %S would yield 28.7808. The question is: what should %s print for 123 microfortnights? I think the appropriate answer is 1487808. That is: while %S prints in seconds modulo 60, %s prints in the unit that would avoid any decimals - in this case in units of 100µs - and without any modulo. And then explicitly providing a precision would affect the number of “decimal” points that are present. This makes %.0s always seconds since epoch, regardless of underlying precision.

That is, a table of examples for formatting `sys_time(microfortnights(123))` would be:

| specifier | example                    |
|-----------|----------------------------|
| %S        | 28.7808 (C++20 status quo) |
| %.0S      | 28                         |
| %.3S      | 28.780                     |
| %.6S      | 28.780800                  |
| %s        | 1487808                    |
| %.0s      | 148                        |
| %.3s      | 148780                     |
| %.4s      | 1487808                    |
| %.6s      | 148780800                  |

## § 4.4 %Q and %q

Separate from the question of what %S and %s should do: are there any other specifiers that we should add? One that I have not noted here is a specifier to simply format `tp.time_since_epoch().count()`. We have such a thing for durations (%Q) but not for `time_points`. In the proposal right now, %s achieves this for the SI units - but not for microfortnights.

When I originally set out to write this paper, my intent was to propose %Q. But given the uniformity of treating %s as (sub)seconds since epoch, that seemed like a better choice to handle formatting nanoseconds since epoch. This makes %Q for `time_point` seem less motivated. But I think we should consider extending %Q to work for `time_point` for broadly the reasons described above.

## § 4.5 Implementation Experience

I've implemented part of this in a fork of `{fmt}`, you can find a diff [here](#). That just implements the suggested changes to %S and the addition of %s, but that's basically all the work — %f is just part of the formatting of %S, and %Q and %q simply treat the `time_point` as its underlying duration.

## § 5 Wording

Add f and s to the options for *type* and add support for precision modifiers in 29.12 [\[time.format\]](#):

```
modifier: one of
- E 0
```

```

+ E 0 tp-precision

+ tp-precision:
+   •opt nonnegative-integer
+   •opt { arg-idopt }

type: one of
- a A b B c C d D e F g G h H I j m M n
+ a A b B c C d D e F f g G h H I j m M n
- p q Q r R S t T u U V w W x X y Y z Z %
+ p q Q r R S s t T u U V w W x X y Y z Z %

```

The rest of the wording adds and modifies entries in the conversion specifier table in 29.12 [\[time.format\]](#).

## § 5.1 The %f specifier

| Specifier | Replacement                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %f        | Sub-seconds as a decimal number. The format is a decimal floating-point number with a fixed format and precision matching that of the precision of the input (or to the <i>tp-precision</i> if provided or otherwise to microseconds precision if the conversion to floating-point decimal seconds cannot be made within 18 fractional digits). The localized decimal point is included if the <i>.</i> appears in the <i>tp-precision</i> and the precision is non-zero. |

## § 5.2 The %Q and %q specifiers

| Specifier | Replacement                                                                                                                                                                                                     |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %q        | The duration's unit suffix as specified in 29.5.11 <a href="#">[time.duration.io]</a> . If the type being formatted is a specialization of <i>time_point</i> , then the unit suffix of the underlying duration. |
| %Q        | The duration's or <i>time_point</i> 's numeric value (as if extracted via <i>.count()</i> or <i>.time_since_epoch().count()</i> , respectively)                                                                 |

## § 5.3 The %S, %s, and %T specifiers

| Specifier | Replacement                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %S        | Seconds as a decimal number. If the number of seconds is less than 10, the result is prefixed with 0. If the precision of the input cannot be exactly represented with seconds, then the format is a decimal floating-point number with a fixed format and a precision matching that of the precision of the input (or to a microseconds precision if the conversion to floating-point decimal seconds cannot be made within 18 |



| Specifier | Replacement                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           | fractional digits). The character for the decimal point is localized according to the locale. The modified command %OS produces the locale's alternative representation. The modified command %tp-precisionS instead uses tp-precision as the precision for the input. The tp-precision must include a .                                                                                             |
| %S        | Duration since epoch as a decimal number in the precision of the input (or in microseconds if the conversion to floating-point decimal seconds cannot be made within 18 fractional digits). The modified command %tp-precisions instead uses tp-precision as the precision of the input. The localized decimal point is included if the . appears in the tp-precision and the precision is non-zero. |
| %T        | Equivalent to %H:%M:%S. The modified command %tp-precisionT is equivalent to %H:%M:%tp-precisionS.                                                                                                                                                                                                                                                                                                   |

## § 5.4 Feature-test Macro

Since the chrono formatters were still handled by \_\_cpp\_lib\_format, let's just bump that one in 17.3.2 [\[version.syn\]](#):

```
- #define __cpp_lib_format 202311L // also in <format>
+ #define __cpp_lib_format 2024XXL // also in <format>
```

## § 6 References

[P0355R0] Howard Hinnant. 2016-05-30. Extending <chrono> to Calendars and Time Zones.

<https://wg21.link/p0355r0>

[P0355R7] Howard E. Hinnant, Tomasz Kamiński. 2018-03-16. Extending <chrono> to Calendars and Time Zones.

<https://wg21.link/p0355r7>

[P1361R2] Victor Zverovich, Daniela Engert, Howard E. Hinnant. 2019-07-19. Integration of chrono with text formatting.

<https://wg21.link/p1361r2>

[P2945R0] Barry Revzin. 2023-07-14. Additional format specifiers for time\_point.

<https://wg21.link/p2945r0>

1. This probably already exists in the literature under a much more suitable name, so I'm hoping by giving it a bad name somebody simply points me to the correct one later. ↩
2. You could argue that it doesn't actually differ from strftime in the sense that in both cases, %S formats all the sub-minute time - it's just that C did not have any subsecond precision. I don't find this argument particularly compelling - %S went from always printing a two-digit integer number of seconds to printing decimals. ↩